PLC Plus — Technical Specification

Nov 4, 2026 · @Ron Eaglin

1. Purpose and scope

PLC Plus is a Windows desktop application that takes a learner from electrical and logic prerequisites through advanced PLC programming, using simulations in place of real hardware. Hands-on work with real PLCs stays in the lab. PLC Plus is the self-paced home component that prepares learners for it.

Primary users

- **Students:** learn at home, practice in simulations, track their own progress, and generate PDF progress reports to hand in to an instructor.
- **Faculty (grant training cohort):** use the same tool to build their own PLC competence before teaching it.
- **Instructors:** receive PDF progress reports through their usual channel, such as an LMS assignment, and set their own submission requirements (Section 9).

In scope

- Guided curriculum from basics to advanced topics, with simulations built into each lesson
- PLC simulation engine, starting with generic IEC 61131-3 and designed so vendor dialects can be added later
- Local, offline progress tracking for multiple learners per machine, with PIN-protected, portable progress files
- PDF progress reports at submission points, saved by the learner and handed in through the instructor's chosen channel
- A reusable core framework that can host other technical subjects later

Out of scope (for now)

- Connecting to or programming real PLC hardware
- Replacing vendor IDEs (Studio 5000, TIA Portal) or certifying vendor skills
- Grade-book authority. PLC Plus reports progress; the instructor and the LMS remain the record of grades.

2. Requirements

Three requirements are non-negotiable. All learning works fully offline. Progress leaves the app only as files the learner chooses to save and hand in. One installation supports many learners, each with a PIN-protected, portable progress file.

Functional requirements

ID	Requirement	Phase
FR-1	Guided curriculum from prerequisites to advanced PLC programming, with prerequisites enforced or recommended	1
FR-2	Ladder Diagram editor and simulator using generic IEC 61131-3 semantics	1
FR-3	Animated process simulations (virtual machines) wired to the program's I/O	1
FR-4	Auto-graded challenges that run the learner's program against scripted scenarios	1
FR-5	Progress tracking: completion, attempts, scores, time on task, hints, mastery per objective	1
FR-6	Learner dashboard showing progress through the whole path and the status of each submission point	1
FR-7	Generate a PDF progress report at each submission point, with learner details and exercise-level performance	1
FR-8	Learner saves the PDF and submits it through whatever channel the instructor specifies (for example, an LMS assignment upload)	1
FR-9	Multiple learners on one installation: select your name from the list and enter a PIN to unlock your progress	1
FR-10	Export and import a learner's progress file so work can continue on another machine, including a personal PC	1
FR-11	Optional report verifier that confirms a PDF report is authentic and unedited	2
FR-12	Structured Text and Sequential Function Chart editors	3
FR-13	Switchable vendor dialects for advanced modules	3

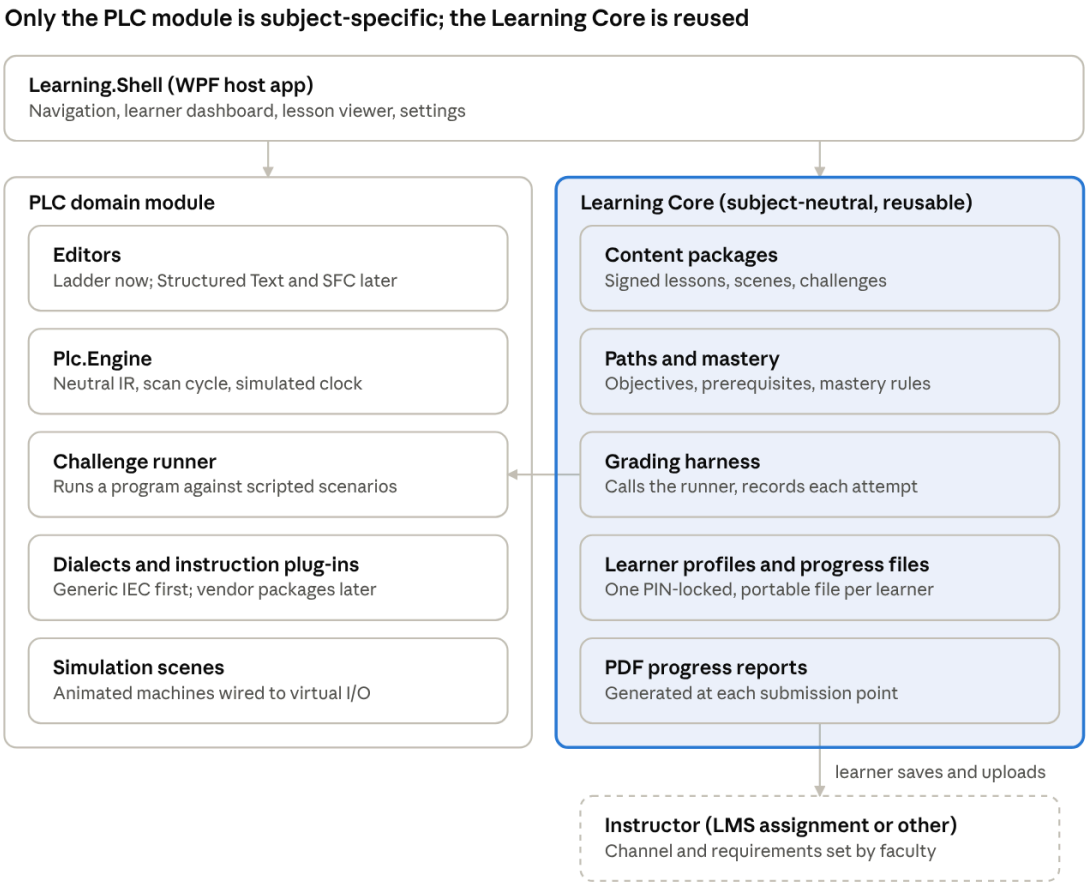
Non-functional requirements

ID	Requirement
NFR-1	Fully usable offline. No account or internet connection is required.
NFR-2	The app makes no network calls with learner data. Data leaves only in files the learner saves.
NFR-3	Simulation timing is deterministic and repeatable, so a given program and scenario always grade the same way.

ID	Requirement
NFR-4	Content (lessons, scenes, challenges, submission points) is data, not code, so it can be updated without rebuilding the app.
NFR-5	The core framework is free of PLC-specific code so it can be reused for other subjects.
NFR-6	Accessibility: keyboard navigation, screen-reader labels, scalable text, color-safe indicators, and tagged PDF reports (Section 508 / WCAG 2.1 AA as the target).
NFR-7	Runs on typical student laptops: Windows 10/11, 8 GB RAM, integrated graphics.
NFR-8	PDF reports are tamper-evident, and progress files are encrypted at rest (Section 10).
NFR-9	A shared lab PC works whether students share one Windows login or use separate accounts.

3. Architecture overview

PLC Plus is two layers inside one host app. A subject-neutral **Learning Core** handles content, mastery, progress, and submission. A **PLC domain module** supplies the editors, the engine, and the simulations. Everything that varies (dialects, scene components, report formats) loads as a signed plug-in.



PLC Plus architecture · host, two layers, external receiver

The only link between the two layers is the challenge-runner contract. The core asks the module to grade work and gets results back, which is what keeps the core reusable (Section 12). The dashed box sits outside the app. The learner carries the PDF report there through whatever channel the instructor sets (Section 9).

4. PLC simulation engine

The engine runs every program from one vendor-neutral internal model. Vendor differences live in swappable dialect packages, so adding Rockwell or Siemens later does not change the engine.

4.1 Neutral program model

- A **project** holds tags (variables), tasks, and program units (programs, functions, function blocks).
- Every editor (ladder, ST, SFC) compiles to the same **intermediate representation (IR)**: a tree of typed instruction nodes that reference tags.
- The IR is serialized as JSON. That makes projects easy to diff and save, and lets the grader inspect them.
- Data types are IEC elementary types (BOOL, INT, DINT, REAL, TIME, and so on), plus arrays and structures. Dialects map their own types onto these.

4.2 Scan cycle

Each scan follows the standard PLC order:

1. Copy virtual inputs from the simulation scene into the input image
2. Run program units in task order and evaluate the IR
3. Copy the output image to the scene's virtual outputs
4. Run housekeeping: timers, diagnostics, trace capture

Key rules:

- **Simulated clock.** The engine advances a virtual clock by a fixed scan time (default 10 ms, configurable), not by wall-clock time. That makes runs deterministic (NFR-3) and lets the grader run scenarios faster than real time.
- **Run modes:** run, pause, single-scan step, and run-to-breakpoint.
- **Forcing and live view:** learners can force I/O and watch tag values and rung power flow live.
- **Faults:** the engine detects watchdog overruns, math faults (such as divide by zero), and out-of-range indexes, and raises them the way a real PLC would, which supports troubleshooting lessons.

4.3 Instruction plug-ins

Each instruction is a self-contained definition with:

- **Signature:** inputs, outputs, parameters, and types
- **Behavior:** an execute method called once per scan, with access to its own instance state
- **Presentation:** symbol, label, and parameter layout for each editor
- **Documentation:** help text and a link to the lesson that teaches it
- **Tests:** a reference test suite that pins its exact semantics

The Phase 1 IEC set covers contacts and coils (NO, NC, set/reset, positive and negative edge), TON/TOF/TP, CTU/CTD/CTUD, comparison, arithmetic, MOVE, and bit logic. Later phases add data handling, sequencers, scaling, and PID.

4.4 Dialects

A dialect is a package that defines addressing and tag syntax, instruction names and symbols, and **behavioral variants** where vendors actually differ. One example: the Allen-Bradley TON with `.PRE`/`.ACC`/`.DN` and a separate retentive RTO behaves differently from the IEC TON with `PT`/`ET`/`Q`.

- Phase 1 ships only **Generic IEC 61131-3**.
- The project file records its dialect. Lessons are tagged either vendor-neutral or with a variant per dialect.
- Converting a project between dialects is not supported unless every instruction it uses has a defined mapping. Otherwise the learner rebuilds it.

4.5 Verification

The engine gets its own automated test suite: per-instruction semantics, scan-order cases, and timer edge cases. Before release, results are compared against a real PLC in the lab for a reference set of programs.

5. Editors and simulation scenes

The Phase 1 editor is Ladder Diagram, the language most US industrial programs use. Every editor is a view onto the shared IR, so adding ST and SFC later does not touch the engine.

5.1 Ladder editor

- Drag-and-drop or keyboard-driven placement of contacts, coils, and function blocks on rungs, with parallel branches
- Tag autocomplete and declaration on first use, with type checking as you type
- Live mode: energized paths are highlighted, live values sit beside each instruction, and forced values are clearly marked
- Rung comments, undo/redo, and copy/paste of rungs
- Validation messages that point to the exact rung and element, written for beginners

5.2 Later editors

- **Structured Text (Phase 3):** code editor with syntax highlighting, compiled to the IR
- **Sequential Function Chart (Phase 3):** steps, transitions, and actions, with the active step shown live
- **Function Block Diagram:** optional and low priority

5.3 Simulation scenes

A scene is a 2D animated machine or process wired to the PLC's virtual I/O. Scenes are assembled from a library of reusable components:

Component type	Examples
Operator inputs	Push buttons, selector switches, e-stop, thumbwheel
Discrete sensors	Limit switch, photo-eye, proximity sensor, float switch
Analog sensors	Level, temperature, pressure, flow
Actuators	Motor (forward/reverse), solenoid valve, cylinder, heater, pump
Indicators	Pilot lights, stack light, numeric display, horn
Process elements	Conveyor, tank, box/part spawner, diverter, elevator car

- Each component exposes typed I/O points. A scene file (JSON) lists its components, their layout, physics parameters (fill rate, travel time), and an I/O map to PLC addresses.
- Scene physics updates on the same simulated clock as the engine, so the whole system stays deterministic.
- Scenes can inject faults for troubleshooting lessons: a stuck sensor, a failed motor, or a wiring error.

Phase 1 scene set: pilot light and push-button station, motor start/stop with seal-in, traffic light, garage door, tank fill and drain, and a conveyor sorter.

6. Curriculum and content model

All curriculum is delivered as versioned **content packages** (data files plus media) that the app loads at runtime. Course content can then change without a new software release (NFR-4).

6.1 Content hierarchy

1. **Learning path:** the full PLC Plus track
    a. **Unit:** a major topic, such as Timers and Counters
        i. **Lesson:** one sitting, typically 20–45 minutes
            1. **Activity:** a reading or explanation page, a guided simulation, a quiz, or a graded challenge

Each unit and lesson declares its **learning objectives** (stable IDs, so they can align with grant and course outcomes), its **prerequisites**, an estimated time, and the dialects it supports.

Submission points are also declared in the learning path, normally at the end of each module or unit. Each one names the exercises it covers and what counts as complete, and reaching it triggers a PDF progress report (Section 9).

6.2 Learning path

Unit	Topics	Phase
1. Electrical foundations	Voltage, current, circuits, AC/DC, safety	1
2. Logic and number systems	Binary, hex, BCD, Boolean logic, truth tables	1
3. Relay control	Relays, contactors, relay ladder diagrams, seal-in circuits	1
4. PLC fundamentals	Hardware, I/O modules, sensors, scan cycle, tags and addressing	1
5. Basic ladder programming	Contacts, coils, latching, edge detection	1
6. Timers and counters	TON, TOF, TP, retentive timing, counters, cascading	2
7. Data and math	Compare, math, move, data types, arrays	2
8. Sequencing and state machines	Step logic, sequencers, state machines, intro to SFC	2
9. Analog and process control	Scaling, alarms, PID concepts and tuning	3
10. Structured programming	Structured Text, function blocks, program organization	3
11. Troubleshooting	Diagnostics, fault finding in faulted scenes	2–3

Unit	Topics	Phase
12. Industrial systems	HMI concepts, networking basics, safety systems, vendor dialects	3

6.3 Package format

- A package is a folder or zip with a `manifest.json` (ID, version, title, dependencies, minimum app version), lesson files in Markdown with front-matter, scene files, challenge files, and media.
- Lessons can embed live widgets such as a mini simulation, a truth-table exercise, or an interactive diagram, using a small tag syntax.
- Packages are signed, so the app can verify they came from an approved author.
- Packages can be updated independently. Progress keys off stable objective and activity IDs, so a content update does not wipe learner progress.

6.4 Authoring

Phase 1 authoring is plain files in a Git repository, with a validator CLI that checks links, IDs, and challenge definitions. That fits an AI-assisted content workflow, and PreseMaker-style generation could feed it. A graphical authoring tool is a possible later addition.

7. Assessment and auto-grading

Challenges are graded by what the learner's program does, not by how it looks. A deterministic test harness drives the scene through scripted scenarios and checks the outputs.

7.1 Challenge definition

Each challenge file contains:

- **Brief:** the task statement, the scene, and the I/O list given to the learner
- **Constraints (optional):** allowed or required instructions, or a maximum rung count
- **Scenarios:** timed sequences of input events with expected outputs. Example: "press Start at t = 0; Motor is ON by t = 0.1 s; press Stop at t = 5 s; Motor is OFF by t = 5.1 s."
- **Assertions:** output states, tag values, timing windows, ordering (A before B), and invariants that must never be broken ("the fill valve and drain valve are never both open")
- **Scoring:** points per scenario and an overall pass threshold
- **Hints:** tiered hints that unlock after failed attempts. Hint use is logged.

7.2 Grading run

1. The learner's project is compiled and checked against the constraints.
2. Each scenario runs headless at accelerated speed on the simulated clock, from a clean reset.
3. Results are reported per scenario, with the failing assertion and the time it failed. The learner can replay the failure visually with a trace.
4. The attempt is recorded: score, time, the hints used, and a hash of the submitted program.

Hidden scenarios (not shown to the learner) discourage hard-coding to the visible tests.

7.3 Other assessment types

Multiple choice, numeric answers (with tolerance), truth-table fill-in, match-the-symbol, and "find the fault" tasks in a faulted scene.

7.4 Mastery

- Each objective has a mastery rule, for example "pass two linked challenges at 80% or higher."
- An objective's state is not started, in progress, mastered, or needs review. "Needs review" can be triggered by a later failure on a dependent objective.
- A unit is complete when all of its required objectives are mastered. Instructors set any stricter requirements in their own assignment instructions, and the PDF report gives them the detail to check against (Section 9).

8. Learner profiles and progress files

One installation serves many learners. Each learner's progress lives in a single portable **progress file**, protected by a PIN. That lets students share a lab PC and also carry their work to other machines, including their own.

8.1 Choosing a learner and entering the PIN

1. The start screen lists every learner who has a progress file on this machine, showing their name and when they last used it.
2. The learner selects their name and enters a 4–6 digit PIN.
3. The progress file unlocks, and learning picks up where they left off.

Other rules:

- **New learner:** enter a display name, an optional student ID, and a PIN. The app then shows a one-time **recovery code** for the learner to write down or print.
- **Forgotten PIN:** the recovery code resets it. Without the recovery code, the file cannot be opened. There is no server, so there is no other way back in.
- **Wrong PINs:** after 5 failed attempts the app makes the learner wait, and the wait grows with each further failure.
- **Auto-lock:** the session locks after a period of no activity (default 15 minutes) and whenever someone chooses Switch Learner.
- **Remove from this machine:** requires the PIN, and the app offers to export the file first.

8.2 The progress file

- Each file holds one learner's complete progress: an encrypted SQLite database in a single `.plcprogress` file.
- A small unencrypted header (display name, profile ID, last modified, app version) lets the start screen list learners without unlocking anything.
- By default, progress files are stored in a shared folder visible to every Windows account on the PC (`%ProgramData%\PLCPlus\Learners`). That way, lab PCs work whether students share one login or use their own. The location can be changed in settings.

8.3 Working on more than one machine

- **Export:** saves the progress file anywhere, such as a USB drive, a cloud folder, or email.
- **Import (Add learner from file):** adds an exported file to this machine's list. The same PIN unlocks it.
- **Two copies that have diverged:** if the learner imports a file and this machine already has a copy, the app merges them rather than overwriting. Attempts are append-only records with unique IDs, so the merge combines the attempts from both copies and recalculates mastery. Nothing is lost.
- Each file records which machine last saved it and when, and the app shows this on import so the learner can confirm they have the right copy.

8.4 Data inside the progress file

Table	Key fields	Notes
LearnerProfile	ProfileId (GUID), DisplayName, StudentId (optional), PinVerifier, RecoveryVerifier	Only verifiers are stored, never the PIN itself
ContentState	PackageId, Version	Content versions the learner has used
ActivityAttempt	AttemptId, ActivityId, StartedUtc, EndedUtc, Score, Passed, HintsUsed, ProgramHash, MachineId	Append-only, which makes merging safe
ObjectiveState	ObjectiveId, State, MasteredUtc, Evidence	Recalculated from attempts
EventLog	EventId, TimestampUtc, Type, PayloadJson	Fine-grained learning events in an xAPI-style shape, for later analytics or research
SubmissionRecord	SubmissionPointId, ReportId, GeneratedUtc, ReportHash, Complete	Every PDF report generated (Section 9)

8.5 Learner dashboard

- The path map shows each unit and lesson as locked, available, in progress, or mastered.
- **Submission points** are shown as not reached, ready to generate, or generated (with the date).
- An objectives view lists evidence for each objective, and a "review suggested" list highlights topics to revisit.
- Time on task and progress totals are always visible.

9. Progress submission (PDF reports at submission points)

At each submission point, PLC Plus generates a PDF progress report. The learner saves it and hands it in however the instructor specifies, typically by uploading it to an LMS assignment. The software never sends anything itself. Instructors keep full control of what is due, when, and through which channel.

9.1 Submission points

- Submission points are declared in the curriculum (Section 6), normally at the end of each module or unit. Each one lists the exercises it covers and what counts as complete.
- When a learner reaches a submission point, the app shows that a report is ready to generate.
- A learner can generate a report even if the work is incomplete. The report is then marked **Incomplete** and lists what is missing, and the instructor decides whether to accept it.
- Reports for earlier submission points can be regenerated at any time. The learner can also generate an on-demand **full progress report** covering everything to date.

9.2 Report contents

Section	Contents
Cover	Learner name, student ID (optional), course and section (optional, typed by the learner), submission point, completion status, generated date and time, report ID
Summary	Percent complete, objectives mastered, total time on task, overall score
Exercise detail	For each exercise: status, best score, number of attempts, time on task, hints used, date completed, and pass or fail for each test scenario
Program evidence (optional)	Printouts of the learner's passing ladder programs for graded challenges, so the instructor can see how the problem was solved
Verification	Report ID, app and content versions, a verification code, and a QR code linking to that code (Section 10)

9.3 Generating and saving

- The learner saves the report through a standard Save dialog, so it can go to the desktop, a USB drive, or a cloud folder.
- Default file name: `PLCPlus_<LastName>_<SubmissionPoint>_<yyyy-MM-dd>.pdf`, which instructors can sort easily in an LMS.
- The output is PDF/A for long-term readability, and tagged PDF for screen readers.
- Every generated report is logged in the learner's submission history (report ID, date, hash), so a learner can show exactly what they produced.

9.4 The instructor's side

- No software is needed: instructors read the PDF in the LMS or any PDF viewer.
- Instructors set requirements in their own assignment instructions, such as which submission point is due, the deadline, and any minimum score.
- An optional **report verifier** (a small companion tool) checks that a PDF matches its verification code and hasn't been edited.

9.5 Keeping it extensible

The report is built from an internal, format-neutral progress snapshot, which is then rendered to PDF. Other outputs (for example CSV for gradebook import, or a direct LMS connection) can be added later as new renderers without changing the progress data or the submission-point rules.

10. Privacy, security, and integrity

PLC Plus collects nothing and sends nothing. Each learner's data stays in their own encrypted progress file, and the PDF reports they choose to hand in are the only thing that reaches an instructor. Once submitted for a course, a report is an education record under FERPA, held in the instructor's LMS.

Privacy

- No telemetry, analytics, or crash reporting without opt-in consent
- The cover page shows only the identity fields the learner entered. Student ID and course are optional.
- Research use of event data requires IRB approval and separate consent, and would use exported data provided voluntarily

PIN and progress-file protection

- Each progress file is encrypted with a key derived from the learner's PIN using a slow key-derivation function (PBKDF2 or Argon2). A second copy of the key is wrapped with the recovery code.
- The PIN is a convenience lock for shared machines, meant to stop classmates opening each other's files. A 4–6 digit PIN cannot withstand a determined offline attack on a copied file, and the documentation says so plainly.
- The escalating delays after failed attempts (Section 8.1) slow down guessing inside the app.

Report integrity

- Each PDF report gets a SHA-256 hash of its contents, signed with a per-installation key pair that is created on first run.
- The verification code and QR code on the report let the report verifier detect any edit made after export.
- Attempts store a hash of the program that passed. When a report includes program evidence, the instructor can re-run that program through the same grader.
- Honest limit: a local, offline app cannot stop a determined learner from tampering with their own machine. For high-stakes assessment, use the lab, or a proctored challenge in which the instructor supplies the hidden scenarios.

Security

- Content packages and plug-ins are signed, and the app rejects anything unsigned or signed by an untrusted key
- The installer and updates are code-signed

11. Technology stack, deployment, and updates

The recommended stack is C# on .NET 10 (LTS) with WPF. It is mature, matches existing Windows desktop experience, and keeps the engine in a plain class library that has no UI dependency.

Area	Choice	Reason
Runtime	.NET 10 LTS	Long-term support across the grant period
UI shell	WPF with MVVM (CommunityToolkit.Mvvm)	Mature, well-known, good accessibility support
Ladder editor and scenes	SkiaSharp canvas hosted in WPF	Fast 2D drawing for rungs and animated scenes
Structured Text editor	AvalonEdit	Syntax highlighting and folding
Lesson rendering	Markdig (Markdown to WPF FlowDocument), or WebView2 for rich content	Content stays plain text
Local storage	SQLite with SQLCipher encryption (Microsoft.Data.Sqlite + Dapper)	One encrypted file per learner, easy to export and merge
Serialization	System.Text.Json with versioned schemas	Projects, scenes, challenges, packages
Plug-ins	Signed assemblies loaded through AssemblyLoadContext	Dialects, scene components, report formats
Testing	xUnit, plus golden-file tests for the engine	Pins PLC semantics
Installer and updates	Velopack (or MSIX), code-signed	Per-user install with no admin rights, and delta updates

Reports and security libraries: QuestPDF for the PDF reports (PDF/A output, with ladder printouts drawn by the same SkiaSharp renderer as the editor), QRCoder for verification QR codes, and .NET's built-in cryptography for key derivation, hashing, and signing.

Solution layout

- `Learning.Core`: content, progress, mastery, and submission framework, with nothing PLC-specific
- `Learning.Shell`: the WPF host app (navigation, dashboard, lesson viewer)
- `Plc.Engine`: IR, scan cycle, and instruction runtime, with no UI
- `Plc.Dialects.Iec`, and later `Plc.Dialects.Rockwell` and `Plc.Dialects.Siemens`
- `Plc.Editors`: the ladder editor, and later ST and SFC
- `Plc.Scenes`: the scene runtime and component library

- `Plc.Grading`: the challenge harness
- `Reports.Pdf`: the progress report renderer (other formats later)
- `Tools.ContentValidator` and `Tools.ReportVerifier`

Deployment: a per-user install of about 150 MB or less, plus content packages. Content updates download separately from app updates. A lab mode supports a machine-wide install for shared lab PCs, with progress files kept in a shared folder (Section 8.2).

12. Framework reuse for other subjects

About half of the system has nothing to do with PLCs. It is built once as the **Learning Core** framework, and each subject plugs in as a **domain module**.

The Learning Core provides:

- The content package format, loader, and signing
- Learning paths, objectives, prerequisites, and mastery rules
- The lesson viewer and embeddable-widget host
- Generic assessments (quiz, numeric, matching)
- The challenge harness contract: run the learner's work against scenarios and report assertions
- The progress database, event log, dashboard, and backup
- Learner profiles, PIN lock, portable progress files, and PDF reports at submission points

A domain module provides:

- Its workspace editors (for PLC Plus, the ladder, ST, and SFC editors)
- Its simulation runtime and scene components
- An implementation of `IChallengeRunner` so the core harness can grade its challenges
- Its own widgets for lesson pages

Candidate future modules: robotics fundamentals (aligned with the Kinova curriculum), motor controls and electrical wiring, pneumatics and hydraulics, sensors and instrumentation, and stormwater or hydrology modeling.

Rule: `Learning.Core` must never reference a `Plc.*` assembly. This is checked in CI with an architecture test (NFR-5).

13. Phased delivery plan and open decisions

Delivery runs in three phases. Each phase ends at a gate that has to be met before the next phase starts. Dates still need to be aligned with the grant timeline.

Phase	Delivers	Gate to exit
1. Core and foundations	Learning Core, IEC engine with the basic instruction set, ladder editor, 6 scenes, grading harness, learner profiles with PIN, portable progress files, dashboard, PDF reports at submission points, Units 1–5	Engine test suite passes; matches a lab PLC on the reference programs; pilot with the faculty training cohort

Phase	Delivers	Gate to exit
2. Intermediate and submission	Units 6–8 and 11, more scenes, fault injection, report verifier tool, PDF report refinements from the faculty pilot	Student pilot in one course section, with PDF reports submitted through the LMS and reviewed end to end
3. Advanced and dialects	Units 9–10 and 12, ST and SFC editors, first vendor dialect, optional extra report formats (such as CSV or LMS integration), research analytics	Advanced modules validated in the lab on the target vendor's hardware

Open decisions

- ☐ Exact report contents and layout instructors need (review with the faculty training cohort), and where submission points fall in each unit
- ☐ Which vendor dialect comes first for the advanced modules (depends on the lab hardware)
- ☐ Identity fields on the report cover (name, student ID, college email)
- ☐ Where progress files live on lab PCs, and whether lab machines are reimaged between terms (this affects the shared folder)
- ☐ Whether learning-event data will be used for research (needs IRB)
- ☐ Licensing and distribution: free to grant participants, open source, or commercial later
- ☐ Whether grant outcome statements map one-to-one to objective IDs
- ☐ Accessibility conformance target and testing approach